Precios de acciones, entrenando un agente.

Descripción

Un área emergente de aplicación es el comercio en el mercado bursátil, donde un trader actúa como un agente de *aprendizaje por refuerzo*, ya que comprar y vender (es decir, actuar) acciones específicas cambia el estado del trader al generar ganancias o pérdidas; es decir, la *recompensa*. La siguiente figura muestra algunas de las acciones más activas al 2026 (como ejemplo):

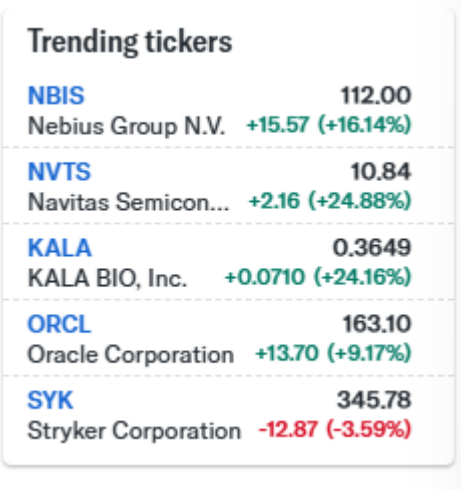

Figura 1. https://finance.yahoo.com/

Ahora, queremos desarrollar un agente inteligente que prediga los precios de las acciones de manera que un trader compre a un precio bajo y venda a un precio alto. Sin embargo, este tipo de predicción no es tan sencilla y depende de varios parámetros, como el **número actual de acciones**, los **precios históricos recientes** y, lo más importante, el **presupuesto disponible** para invertir en compras y ventas.

Los *estados* en esta situación son un vector que contiene información sobre el **presupuesto actual**, **el número actual de acciones** y **un historial reciente de precios de acciones** (los últimos 200 precios). Así que cada estado es un *vector de 202 dimensiones*. Para simplificar, solo hay tres acciones que puede realizar un agente de mercado bursátil: comprar, vender y mantener.

Entonces, tenemos el estado y la acción, ¿qué más se necesita? ¿Política, verdad? Sí, debemos tener una buena política, de modo que a partir de ella se determine qué acción se realizará en un estado. Una política simple puede consistir en las siguientes reglas:

- **Comprar** (es decir, la acción) una acción al precio actual (es decir, el estado) disminuye el presupuesto mientras incrementa el número actual de acciones.
- **Vender** una acción la intercambia por dinero al precio actual por acción.
- **Mantener** no hace ninguna de las anteriores, y realizar esta acción simplemente espera un periodo de tiempo determinado y no produce ninguna recompensa.

Para obtener los precios de las acciones, podemos usar la librería yahoo_finance en Python.

Revisar la documentación de la página oficial para instalar la librería.

Realiza una prueba en el Shell para familiarizarse con el módulo:

```
>>> from yahoo_finance import Share
>>> msoft = Share('MSFT')
>>> print(msoft.get_open())
72.24=
>>> print(msoft.get_price())
72.78
>>> print(msoft.get_trade_datetime())
2017-07-14 20:00:00 UTC+0000
>>>
```

Interpretación:

El precio de las acciones de Microsoft Inc. aumentó, de 72.24 a 72.78, lo que significa un incremento de aproximadamente 7.5%. Sin embargo, este dato pequeño y de solo un día no nos brinda información significativa. Pero, al menos, conocemos el estado actual de esta acción o instrumento en particular.

La siguiente función nos ayuda a obtener los datos históricos de Microsoft Inc.:

```
def get_prices(share_symbol, start_date, end_date,
cache_filename):
    try:
        stock_prices = np.load(cache_filename)
    except IOError:
        share = Share(share_symbol)
        stock_hist = share.get_historical(start_date,
end_date)
        stock_prices = [stock_price['Open'] for
stock_price in stock_hist]
        np.save(cache_filename, stock_prices)
    return stock_prices
```

El método **`get_prices()`** toma varios parámetros, como el símbolo de la acción de un instrumento en el mercado bursátil, la fecha de inicio y la fecha de fin. También querrás especificar y almacenar en caché los datos históricos para evitar descargas repetidas. Una vez que hayas descargado los datos, es momento de graficarlos para obtener información.

La siguiente función nos ayuda a graficar el precio:

```
def plot_prices(prices):
    plt.title('Opening stock prices')
    plt.xlabel('day')
    plt.ylabel('price ($)')
    plt.plot(prices)
    plt.savefig('prices.png')
```

Ahora podemos llamar a estas dos funciones especificando un argumento real de la siguiente manera:

```
if __name__ == '__main__':
    prices = get_prices('MSFT', '2000-07-01', '2017-07-
01', 'historical_stock_prices.npy')
    plot_prices(prices)
```

Aquí he elegido un rango amplio de datos históricos de 17 años para obtener una mejor perspectiva. Ahora, analicemos el resultado de estos datos:

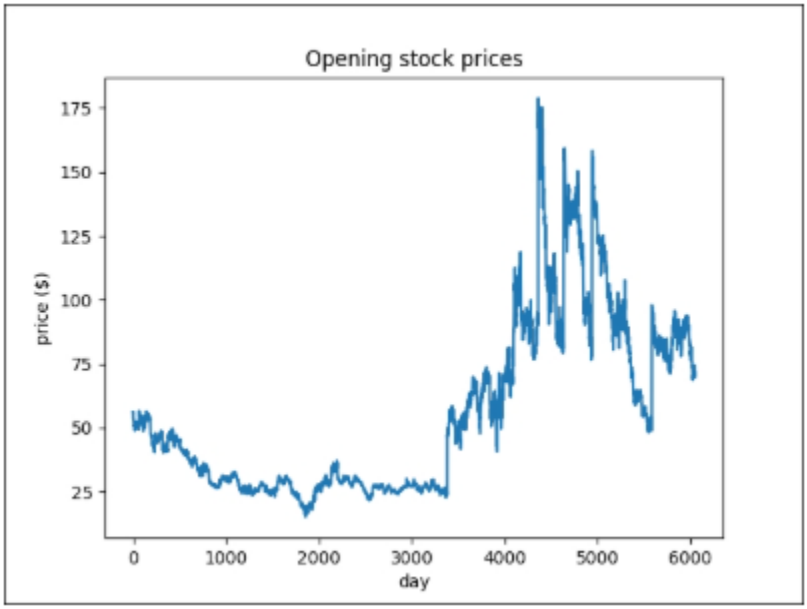

Figura 2. Datos históricos del precio de las acciones de Microsoft Inc. desde 2000 hasta 2017.

Nota: Modificar el rango de las fechas a la actual.

El objetivo es aprender una política que obtenga el mayor valor neto posible mediante el comercio en el mercado de valores. Entonces, ¿qué logrará un agente de comercio al final? La figura muestra una pista:

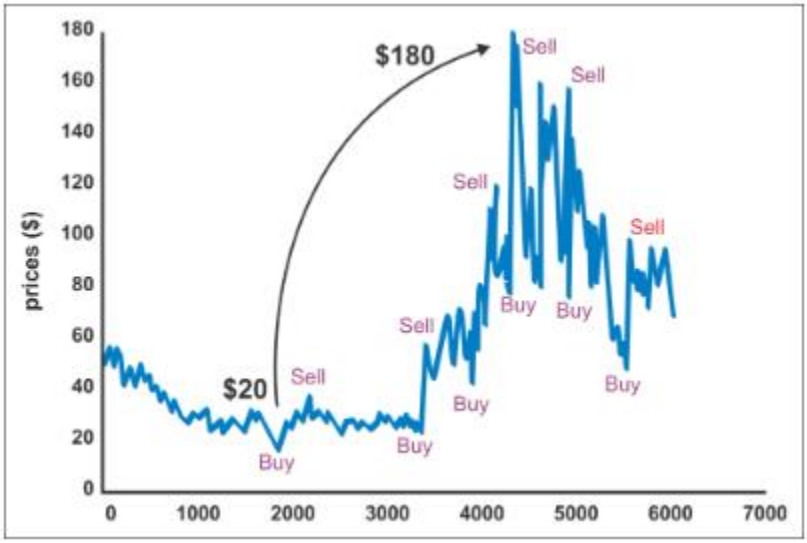

Figura 3. Algo de información y una pista que muestra que, según el precio actual, se pueden obtener hasta $160 de ganancia

Bueno, la Figura 3 muestra que si el agente compra un instrumento determinado con un precio de $20 y vende en un punto máximo, digamos $180, podrá obtener una recompensa de $160, es decir, ganancia.

Entonces, ¿implantar un agente inteligente usando algoritmos de aprendizaje por refuerzo (RL) es una buena idea?

Del ejemplo anterior, hemos visto que, para que un agente RL tenga éxito, necesitamos dos operaciones bien definidas, que son las siguientes:

- Cómo seleccionar una acción
- Cómo mejorar la función de utilidad Q (Q-function)

Para ser más específicos, dado un estado, la política de decisión calculará la siguiente acción a tomar. Por otro lado, se mejora la Q-function a partir de una nueva experiencia de tomar una acción.

Además, la mayoría de los algoritmos de aprendizaje por refuerzo se reducen a solo tres pasos principales: **inferir**, **actuar** y **aprender**. Durante el primer paso, el algoritmo selecciona la mejor acción (a) dada un estado (s) usando el conocimiento que tiene hasta el momento. Luego, ejecuta la acción para descubrir la recompensa (r), así como el siguiente estado (s').

Después, mejora su comprensión del mundo usando el conocimiento recién adquirido (s, r, a, s') como se muestra en la siguiente figura:

$$\text{Infer}(s) \Rightarrow a$$
$$\text{Do}(s,a) \Rightarrow r,\ s'$$
$$\text{Learn}(s,\ r,\ a,\ s')$$

Figura 4. Pasos a realizar para implementar un agente inteligente de predicción de precios de acciones.

Ahora, se comienza a implementar la política de decisión basada en cuál acción se tomará para comprar, vender o mantener una acción. Nuevamente, lo haremos de forma incremental. Primero, crearemos una **política de decisión aleatoria** y **evaluaremos el rendimiento del agente**.

Pero antes, se desarrollará una clase abstracta para poder implementarla adecuadamente:

```
class DecisionPolicy:
    def select_action(self, current_state, step):
        pass
    def update_q(self, state, action, reward,
next_state):
        pass
```

La siguiente tarea que se puede realizar es heredar de esta superclase para implementar una política de decisión aleatoria:

```
class RandomDecisionPolicy(DecisionPolicy):
    def __init__(self, actions):
        self.actions = actions
    def select_action(self, current_state, step):
        action = self.actions[random.randint(0,
len(self.actions) - 1)]
        return action
```

La clase anterior no hizo nada, excepto definir una función llamada **`select_action()`**, que seleccionará aleatoriamente una acción sin siquiera mirar el estado.

Ahora, si deseas usar esta política, puedes ejecutarla con datos reales del precio de acciones. Esta función se encarga de la exploración y explotación en cada intervalo de tiempo, como se muestra en la siguiente figura que forma los estados S1, S2 y S3. La política sugiere una acción a tomar, la cual podemos elegir explotar o, de lo contrario, explorar aleatoriamente otra acción. A medida que obtenemos recompensas por realizar una acción, podemos actualizar la función de política con el tiempo:

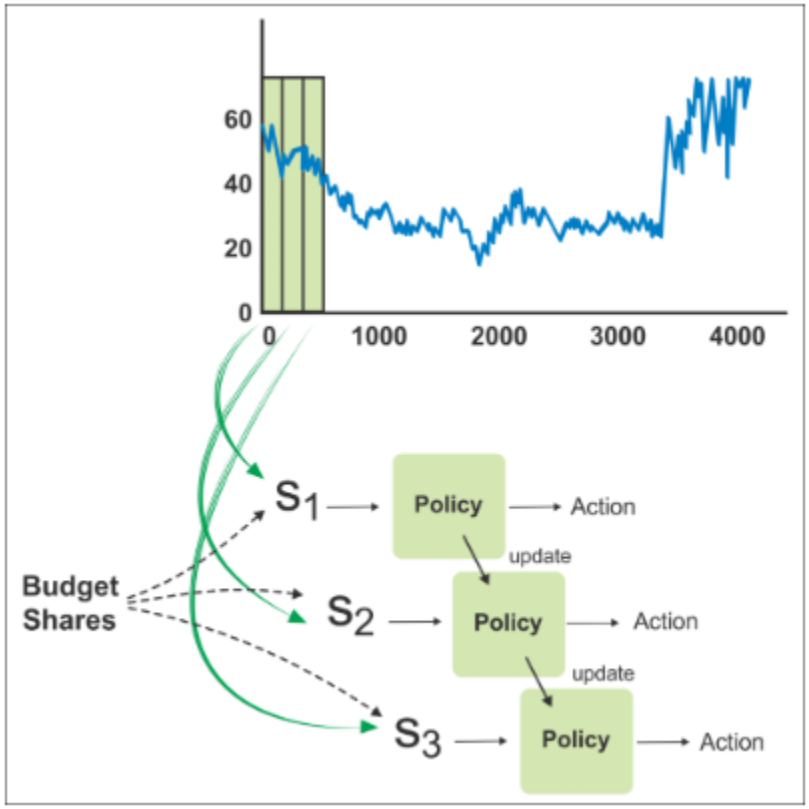

Figura 5. Una ventana deslizante de cierto tamaño que itera a través de los precios de las acciones a lo largo del tiempo

Teniendo la política, es momento de utilizarla para tomar decisiones y evaluar el rendimiento. Ahora, imagina un escenario real —supongamos que estás operando en Forex o en la plataforma ForTrade— entonces puedes recordar que también necesitas calcular el portafolio y la ganancia o pérdida actual, es decir, la recompensa. Normalmente, estas se pueden calcular de la siguiente forma:

```
portfolio = budget + number of stocks * share value
reward = new_portfolio - current_portfolio
```

Al principio, podemos inicializar valores que dependen de calcular el valor neto de un portafolio, donde el estado es un vector de *hist + 2* dimensiones. En nuestro caso, sería de 202 dimensiones. Luego definimos el rango del que tomaremos datos:

> La longitud de los precios seleccionados por la *consulta del usuario – (historia + 1),* ya que comenzamos desde cero, por lo que restamos 1 en su lugar. Luego, deberíamos calcular el valor actualizado del portafolio y, a partir de este, podemos obtener el valor de la recompensa, es decir, la ganancia.

Además, ya hemos definido nuestra política aleatoria, por lo que ahora podemos seleccionar una acción a partir de la política actual. Repetidamente actualizamos los valores del portafolio basándonos en la acción en cada iteración y el nuevo valor del portafolio después de tomar la acción puede calcularse. Luego, necesitamos computar la recompensa al tomar una acción dada en un estado.

No obstante, también necesitamos actualizar la política después de experimentar una acción nueva. Finalmente, calculamos el valor final del portafolio:

```
def run_simulation(policy, initial_budget, initial_num_stocks, prices,
hist, debug=False):
    budget = initial_budget
    num_stocks = initial_num_stocks
    share_value = 0
    transitions = list()
    for i in range(len(prices) - hist - 1):
        if i % 100 == 0:
            print('progress {:.2f}%'.format(float(100*i) / (len(prices)
- hist - 1)))
        current_state = np.asmatrix(np.hstack((prices[i:i+hist], budget,
num_stocks)))
        current_portfolio = budget + num_stocks * share_value
        action = policy.select_action(current_state, i)
        share_value = float(prices[i + hist + 1])
        if action == 'Buy' and budget >= share_value:
            budget -= share_value
            num_stocks += 1
        elif action == 'Sell' and num_stocks > 0:
            budget += share_value
            num_stocks -= 1
        else:
            action = 'Hold'
        new_portfolio = budget + num_stocks * share_value
        reward = new_portfolio - current_portfolio
        next_state = np.asmatrix(np.hstack((prices[i+1:i+hist+1],
budget, num_stocks)))
        transitions.append((current_state, action, reward, next_state))
        policy.update_q(current_state, action, reward, next_state)
    portfolio = budget + num_stocks * share_value
    if debug:
        print('${}\t{} shares'.format(budget, num_stocks))
    return portfolio
```

La simulación anterior predice un resultado algo bueno; sin embargo, produce resultados aleatorios con demasiada frecuencia. Por lo tanto, para obtener una medición más sólida del éxito, ejecutemos la simulación un par de veces y promediemos los resultados. Hacer esto puede tomar un tiempo en completarse, digamos 100 veces, pero los resultados serán más confiables:

```
def run_simulations(policy, budget, num_stocks, prices, hist):
    num_tries = 100
    final_portfolios = list()
    for i in range(num_tries):
        final_portfolio = run_simulation(policy, budget, num_stocks,
prices, hist)
        final_portfolios.append(final_portfolio)
    avg, std = np.mean(final_portfolios), np.std(final_portfolios)
    return avg, std
```

La función anterior calcula el portafolio promedio y la desviación estándar repitiendo la función de simulación 100 veces. Ahora es momento de evaluar nuestro agente anterior. Como ya se mencionó, habrá tres posibles acciones que puede tomar el agente de trading: comprar, vender y mantener. Tenemos un vector de estado con 202 dimensiones y un presupuesto de solo $1,000. Luego, la evaluación procede así:

```
actions = ['Buy', 'Sell', 'Hold']
    hist = 200
    policy = RandomDecisionPolicy(actions)
    budget = 1000.0
    num_stocks = 0
    avg,std=run_simulations(policy,budget,num_stocks,prices, hist)
    print(avg, std)
>>>
1512.87102405 682.427384814
```

El primero es el promedio y el segundo es la desviación estándar del portafolio final. Entonces, nuestro agente de predicción de acciones predice que un trader podría obtener una ganancia de alrededor de **$513.10**. Sin embargo, el problema es que, debido a que utilizamos una política aleatoria, el resultado no es tan confiable. Para ser más específicos, la segunda ejecución probablemente produciría un resultado diferente.

```
>>>
1518.12039077 603.15350649
```

Por lo tanto, debemos desarrollar una política de decisión más robusta. Aquí entra en juego el uso de **Q-Learning basado en redes neuronales** para la política de decisión.

A continuación, veremos un nuevo hiperparámetro epsilon para evitar que la solución se quede atascada al aplicar repetidamente la misma acción una y otra vez. Cuanto menor sea el valor de epsilon, más frecuentemente explorará acciones nuevas de manera aleatoria:

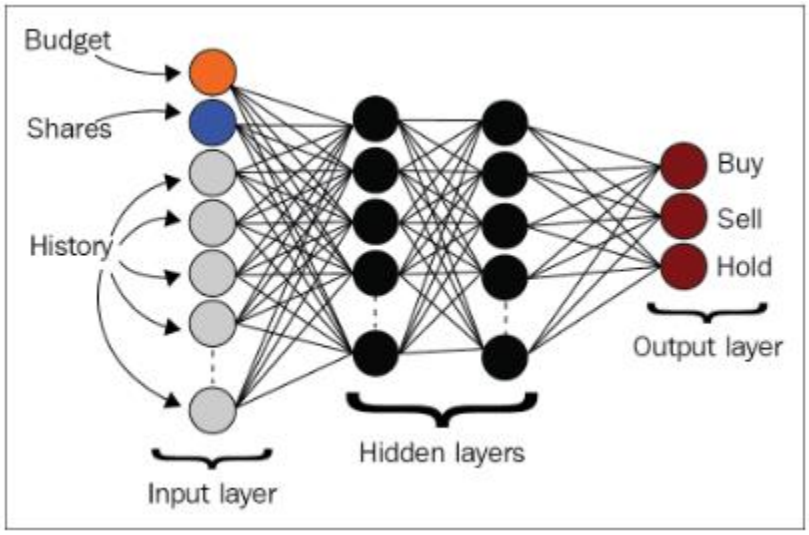

Figura 6. La entrada es el vector de estado con tres salidas, una para cada valor Q por acción — comprar, vender, mantener.

A continuación, se describen las funciones de la clase:

- **`Constructor`:** Esto ayuda a establecer los hiperparámetros de la función Q. También ayuda a configurar el número de nodos ocultos en la red neuronal. Una vez que tenemos estos dos, ayuda a definir los tensores de entrada y salida. Luego define la estructura de la red neuronal. Después, define las operaciones para calcular la función de utilidad. Después utiliza un optimizador para actualizar los

parámetros y minimizar la pérdida, configura la sesión e inicializa las variables.

- **`select_action`:** Esta función explota la mejor opción con probabilidad **1 - epsilon**.
- **`update_q`:** Esto actualiza la función Q al actualizar los parámetros de su modelo.

```
class QLearningDecisionPolicy(DecisionPolicy):
    def __init__(self, actions, input_dim):
        self.epsilon = 0.9
        self.gamma = 0.001
        self.actions = actions
        output_dim = len(actions)
        h1_dim = 200
        self.x = tf.placeholder(tf.float32, [None, input_dim])
        self.y = tf.placeholder(tf.float32, [output_dim])
        W1 = tf.Variable(tf.random_normal([input_dim, h1_dim]))
        b1 = tf.Variable(tf.constant(0.1, shape=[h1_dim]))
        h1 = tf.nn.relu(tf.matmul(self.x, W1) + b1)
        W2 = tf.Variable(tf.random_normal([h1_dim, output_dim]))
        b2 = tf.Variable(tf.constant(0.1, shape=[output_dim]))
        self.q = tf.nn.relu(tf.matmul(h1, W2) + b2)
        loss = tf.square(self.y - self.q)
        self.train_op =
tf.train.GradientDescentOptimizer(0.01).minimize(loss)
        self.sess = tf.Session()
        self.sess.run(tf.initialize_all_variables())
    def select_action(self, current_state, step):
        threshold = min(self.epsilon, step / 1000.)
        if random.random() < threshold:
            # Exploit best option with probability epsilon
            action_q_vals = self.sess.run(self.q, feed_dict=
{self.x: current_state})
            action_idx = np.argmax(action_q_vals)
            action = self.actions[action_idx]
        else:
            # Random option with probability 1 - epsilon
            action = self.actions[random.randint(0,
len(self.actions) - 1)]
        return action
    def update_q(self, state, action, reward, next_state):
        action_q_vals = self.sess.run(self.q, feed_dict={self.x:
state})
        next_action_q_vals = self.sess.run(self.q, feed_dict=
{self.x: next_state})
        next_action_idx = np.argmax(next_action_q_vals)
        action_q_vals[0, next_action_idx] = reward + self.gamma *
next_action_q_vals[0, next_action_idx]

        action_q_vals = np.squeeze(np.asarray(action_q_vals))
        self.sess.run(self.train_op, feed_dict={self.x: state, self.y:
action_q_vals})
```

Karim, R. (2018). *Tensorflow : Powerful predictive analytics with tensorflow : predict valuable insights of your data with tensorflow*. Packt Publishing, Limited.